

OptArrow Design Definition Documentation

1. Module & Class Structure

OptArrow's architecture is based on a modular design that allows for extensibility and robust communication between components.

Core Software Modules

- **SolverManager Module:** Manages interactions with various solver backends, providing interfaces for both Python and Julia solvers.
- **TransportLayer Module:** Facilitates the data communication across the system using Apache Arrow for efficient data transfer.
- **ClientInterface Module:** Handles requests from external clients, such as MATLAB, ensuring compatibility and appropriate routing of optimization problems.

Solver Adapters

Each solver backend, whether Python, Julia, or third-party (e.g., HiGHS, open-source solver), is encapsulated within a dedicated adapter class. These classes interface directly with the `SolverManager`, converting optimization problems into formats suitable for the respective solvers.

Class Dependencies

- The `SolverManager` depends on specific classes within Solver Adapters to decouple solver-specific logic from core system functionalities.
- The `ClientInterface` classes are dependent on the `TransportLayer` to ensure correct data encoding and retrieval.

2. API Contracts & Data Schemas

OptArrow uses a set of well-defined API contracts to ensure consistent and reliable interaction between components.

API Syntax

- APIs offer methods for submitting optimization problems, checking solver status, and retrieving results.
- Each API endpoint is standardized, facilitating requests in JSON format when communicating across different environments.

Data Schema Definitions

- Input data, irrespective of origin, must conform to predefined schemas ensuring that only valid optimization problem specifications are processed.
- The IPC format, facilitated by Apache Arrow, is employed consistently across APIs to

maintain coherence and add secondary checks for data integrity.

3. Data Transformations & Protocol Structures

Arrow IPC Structure

The choice of Apache Arrow as the primary data transportation protocol is justified by its efficient in-memory data management capabilities and its support for multiple language bindings.

- **Data Transformation Process:**

- Input data, initially in Python dictionaries or Julia structures, is serialized into Arrow tables.
- Arrow's IPC format enables direct memory access, reducing latency and power consumed during transfers between different language environments.

Data Transformation Protocols

Protocols define rules for serializing, deserializing and transferring data specifically between OptArrow's core components and the Edge Problem Solver (EPS) environments.

4. Component Internals & Solver Adapters

Component Architecture Exploration

- **SolverManager Internals:** Centralizes decision-making logic regarding which backend solver to deploy based on problem fit criteria (e.g., complexity, type).
- **TransportLayer Internals:** Employs Arrow for creating efficient pipelines between component interfaces, guaranteeing minimal data conversion overhead.

Solver API Interaction & HiGHS Integration

- **Solver Adapters** directly interact with backend solver APIs, treating them as black boxes.
- HiGHS integration within OptArrow is facilitated via a custom adapter, simplifying access to its features through high-level API calls.

5. Error Handling & Edge Cases

Error Trapping Methods

The system employs robust error-handling strategies including:

- Structured exception handling around third-party library calls to catch errors specific to external APIs and solvers.
- Logging mechanisms embedded within `TransportLayer` and `SolverManager` modules to ensure traceability of execution and performance anomalies.

Edge Case Adaptation

Adaptive algorithms within OptArrow manage resource allocation dynamically based on problem complexity and computational demands, offering reliable performance under edge conditions.

6. Detailed Sequence of Operations

Request Processing Sequence

1. **Problem Identification:** Determine the source environment.
2. **Data Preparation:** Format into Arrow IPC for internal use.
3. **Solver Routing:** Select and dispatch to appropriate solver backend.
4. **Computation Cycle:** Solver executes and returns results.
5. **Result Delivery:** Final results are sent back in the originating language format.

Data Routing Paths

- Managed through the `TransportLayer`, which orchestrates messaging between the client interfaces and solver adapters while maintaining state consistency across transactions.

7. Implementation-Level Decisions

Decision-Making Backdrop

- Choices such as Apache Arrow's implementation stemmed from its capacity to support cross-language operations, notwithstanding a formal trial of alternative protocols.
- The preference for HiGHS was informed by its open-source nature and broad applicability, which aligns with OptArrow's design philosophy.

Impact of Chosen Architectural Strategies

- The integration of robust data transport and solver adaptability mechanisms directly contributes to OptArrow's capability to function efficiently across the varied computational environments of its stakeholders. Expanding on architectural governance to include ADRs will enhance long-term maintainability.

The design and components outlined herein ensure that OptArrow meets its primary objectives of efficient and cross-environmental optimization problem-solving, while continuously allowing for improvements in system architecture and functionality based on stakeholder feedback.